

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу «Компьютерная графика»

Основы 3D графики

Вариант: правильный октаэдр

Выполнил: А. А. Устинов

Группа: М8О-307БВ-24

Преподаватель: В. Д. Бахарев

Москва, 2026

1. Условие

Цель работы — познакомиться с основами трёхмерной графики: построением простых 3D-объектов, проекцией трёхмерной сцены на двумерную плоскость экрана, а также научиться работать с матрицами перспективной и ортогографической проекций и матрицами аффинных преобразований.

Вариант задания — построить и вывести на экран правильный октаэдр. Программа должна работать в реальном времени с использованием Vulkan, GLFW и библиотеки интерфейса ImGui и допускать динамическую смену проекции и преобразований объекта.

Выполнены 5 дополнительных заданий:

1. переключение матрицы проекции (ортогографическая и перспективная) в интерфейсе;
2. элементы интерфейса для изменения позиции, поворота и растяжения фигуры;
3. движение и поворот фигуры по сложной траектории, пауза и воспроизведение анимации, изменение скорости и параметров траектории;
4. изменение цвета фигуры с помощью элемента ColorEdit;
5. процедурное назначение цветов вершинам в зависимости от их позиции в локальной системе координат; выбранный в интерфейсе цвет умножается на цвет вершины.

2. Метод решения

2.1. Однородные координаты и аффинные преобразования

Объект описывается один раз в собственной (локальной) системе координат, а затем должен быть размещён в сцене, снят виртуальной камерой и спроецирован на плоскость экрана. Каждое из этих действий — преобразование координат вершины. Обозначения, используемые далее, сведены в таблицу 1.

Таблица 1 – Обозначения в формулах преобразований

Обозначение	Что обозначает
$\mathbf{v}$	вершина до преобразования, $\mathbf{v} = (x, y, z, 1)^T$
$\mathbf{v}'$	та же вершина после преобразования
w	четвёртая координата результата; на неё выполняется деление
m_{ij}	элемент матрицы в строке i и столбце j
$\mathbf{t} = (t_x, t_y, t_z)$	вектор переноса: на сколько сдвинуть фигуру по каждой оси
$\mathbf{s} = (s_x, s_y, s_z)$	коэффициенты растяжения по каждой оси
α, β, γ	углы поворота вокруг осей X, Y и Z соответственно
$T(\mathbf{t})$	матрица переноса
$S(\mathbf{s})$	матрица масштабирования
R_x, R_y, R_z	матрицы поворота вокруг соответствующих осей
M	матрица модели: из локальных координат фигуры в мировые
V	матрица вида: из мировых координат в систему координат камеры
P	матрица проекции: из системы координат камеры в координаты экрана
$\mathbf{e}$	положение камеры в мировых координатах
$\mathbf{c}$	точка, на которую направлена камера
$\mathbf{u}_0$	приблизительное направление «вверх», задаваемое пользователем
$\mathbf{d}, \mathbf{r}, \mathbf{u}$	орты камеры: вперёд, вправо и вверх

Повороты и масштабирование линейны и выражаются матрицей 3×3 . Перенос линейным преобразованием не является: сдвиг на вектор $\mathbf{t}$ нельзя записать как умножение вектора на матрицу 3×3 , поскольку любая такая матрица оставляет начало координат на месте. Поэтому точка записывается в однородных координатах — четырёхмерным вектором $(x, y, z, 1)^T$, а преоб-

разования — матрицами 4×4 .

Действие матрицы на вершину в развёрнутом виде:

$$\begin{pmatrix} x' \\ y' \\ z' \\ w' \end{pmatrix} = \begin{pmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \\ m_{41} & m_{42} & m_{43} & m_{44} \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}, \quad x' = m_{11}x + m_{12}y + m_{13}z + m_{14}. \quad (1)$$

Каждая строка матрицы даёт одну координату результата. Четвёртый столбец умножается на единицу, то есть прибавляется к результату без изменений, — именно это и позволяет выразить перенос. Первые три столбца имеют наглядный смысл: это образы базисных векторов, то есть то, во что переходят единичные векторы осей X , Y и Z после преобразования.

Три базовых преобразования записываются так:

$$T(\mathbf{t}) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad S(\mathbf{s}) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad (2)$$

$$R_y(\beta) = \begin{pmatrix} \cos \beta & 0 & \sin \beta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \beta & 0 & \cos \beta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (3)$$

В матрице переноса $T(\mathbf{t})$ компоненты вектора сдвига стоят в четвёртом столбце и, согласно (1), прибавляются к координатам. В матрице масштабирования $S(\mathbf{s})$ коэффициенты стоят на главной диагонали, поэтому каждая координата умножается на свой множитель независимо от остальных.

В матрице поворота $R_y(\beta)$ изменяются только координаты x и z : поворот вокруг оси Y происходит в плоскости XZ , а координата y остаётся прежней, что и отражает единица во второй строке. Синусы и косинусы возникают здесь не произвольно: при повороте на угол β единичный вектор оси X переходит в точку единичной окружности с координатами $(\cos \beta, \sin \beta)$ — это определение тригонометрических функций, — а единичный вектор

перпендикулярной оси переходит в $(-\sin \beta, \cos \beta)$. Эти две пары и стоят в соответствующих столбцах. Матрицы $R_x(\alpha)$ и $R_z(\gamma)$ строятся так же, но в плоскостях YZ и XY .

Четвёртая координата решает две задачи. Во-первых, как показано выше, делает возможным перенос. Во-вторых, после всех преобразований выполняется перспективное деление: аппаратура вычисляет отношения x'/w' , y'/w' и z'/w' — именно деление на w' даёт эффект перспективы.

Благодаря однородным координатам все преобразования становятся объектами одного типа и перемножаются между собой, поэтому цепочка сворачивается в одну матрицу. Она вычисляется один раз за кадр, а к каждой вершине применяется однократно:

$$\mathbf{v}' = P \cdot V \cdot M \cdot \mathbf{v}. \quad (4)$$

Матрица модели собирается из трёх базовых преобразований:

$$M = T(\mathbf{t}) \cdot R_z(\gamma) R_x(\alpha) R_y(\beta) \cdot S(\mathbf{s}). \quad (5)$$

Порядок существенен: матрицы перемножаются справа налево, поэтому к вершине сначала применяется масштабирование, затем поворот и только потом перенос. При обратном порядке перенос был бы выполнен до поворота, и, поскольку поворот всегда происходит вокруг начала координат, фигура вращалась бы по орбите вокруг центра сцены, а не вокруг собственного центра.

Камеры как отдельной сущности в математике преобразований нет: вместо перемещения наблюдателя выполняется обратное перемещение всей сцены. Матрица вида V строится по положению камеры $\mathbf{e}$, точке наблюдения $\mathbf{s}$ и вектору $\mathbf{u}_0$ через ортонормированный базис камеры:

$$\mathbf{d} = \frac{\mathbf{c} - \mathbf{e}}{\|\mathbf{c} - \mathbf{e}\|}, \quad \mathbf{r} = \frac{\mathbf{d} \times \mathbf{u}_0}{\|\mathbf{d} \times \mathbf{u}_0\|}, \quad \mathbf{u} = \mathbf{r} \times \mathbf{d}. \quad (6)$$

Здесь $\mathbf{d}$ — единичный вектор направления взгляда, полученный нормированием разности между точкой наблюдения и положением камеры; $\mathbf{r}$ — направление «вправо от камеры», перпендикулярное направлению взгляда и

вектору «вверх», для чего используется векторное произведение; $\mathbf{u}$ — уточнённое направление «вверх»: исходный $\mathbf{u}_0$ может не быть перпендикулярен взгляду, поэтому верх пересчитывается как перпендикуляр к двум уже найденным ортам. Нормирование здесь не требуется, так как векторное произведение двух единичных перпендикулярных векторов само единично.

Первые три строки матрицы V составлены из векторов $\mathbf{r}$, $\mathbf{u}$ и $-\mathbf{d}$, а четвёртый столбец содержит положение камеры, спроецированное на эти направления и взятое с обратным знаком, — он переносит сцену так, чтобы камера оказалась в начале координат. Знак минус перед $\mathbf{d}$ отражает принятое соглашение: камера смотрит вдоль отрицательного направления оси Z , поэтому всё, что находится перед ней, получает отрицательную координату z .

2.2. Матрицы проекций

Обозначения, используемые в матрицах проекций, приведены в таблице 2.

Таблица 2 – Обозначения в матрицах проекций

Обозначение	Что обозначает
φ	вертикальный угол обзора камеры (для перспективной проекции)
h	высота видимой области в мировых единицах (для ортографической)
a	отношение сторон окна: ширина, делённая на высоту
n	расстояние до ближней плоскости отсечения
f	расстояние до дальней плоскости отсечения
$k = \text{ctg}(\varphi/2)$	вспомогательный коэффициент, задающий масштаб по вертикали

Перспективная проекция моделирует зрение человека: чем дальше объект, тем меньше он выглядит.

$$P_{\text{persp}} = \begin{pmatrix} \frac{k}{a} & 0 & 0 & 0 \\ 0 & -k & 0 & 0 \\ 0 & 0 & \frac{f}{n-f} & \frac{nf}{n-f} \\ 0 & 0 & -1 & 0 \end{pmatrix}. \quad (7)$$

Коэффициент k возникает из определения угла обзора: на расстоянии, равном единице, в кадр попадает полоса высотой $\text{tg}(\varphi/2)$ в каждую сторону от оси взгляда, а экранные координаты должны лежать в пределах от -1 до 1 . Следовательно, координаты нужно разделить на $\text{tg}(\varphi/2)$, то есть умножить на k .

Назначение строк матрицы (7):

- первая строка задаёт масштаб по горизонтали; деление на a учитывает форму окна, иначе на широком окне изображение растянулось бы;
- вторая строка задаёт масштаб по вертикали; знак минус инвертирует ось Y (см. ниже);
- третья строка переводит координату z из отрезка $[n, f]$ в отрезок $[0, 1]$ — именно эти значения затем сравнивает буфер глубины;
- четвёртая строка — ключевая. Элемент -1 записывает в w' величину $-z$, то есть расстояние от камеры до точки. При последующем делении на w' экранные координаты уменьшаются обратно пропорционально расстоянию, что и даёт перспективное сокращение.

Ортографическая проекция переносит точки на плоскость параллельно, без схождения линий: размер объекта не зависит от расстояния до камеры.

$$P_{\text{ortho}} = \begin{pmatrix} \frac{2}{h a} & 0 & 0 & 0 \\ 0 & -\frac{2}{h} & 0 & 0 \\ 0 & 0 & \frac{1}{n-f} & \frac{n}{n-f} \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (8)$$

Здесь первые две строки ужимают видимую область размером ha на

h мировых единиц в экранный диапазон от -1 до 1 : половина высоты равна $h/2$, поэтому множитель равен $2/h$, а для горизонтали он дополнительно делится на отношение сторон. Третья строка, как и в (7), отвечает за глубину.

Четвёртая строка здесь единичная: w' остаётся равным единице, деления не происходит, и зависимость размера от расстояния исчезает. Таким образом, различие двух проекций сводится к одному элементу матрицы, и задание 1 реализовано выбором функции, строящей P .

Обе формулы отличаются от приводимых в литературе по OpenGL по двум причинам. Во-первых, в Vulkan ось Y в координатах кадра направлена вниз, тогда как в мировой системе координат она направлена вверх; поэтому обе матрицы содержат знак минус во второй строке, иначе изображение оказалось бы перевёрнутым. Во-вторых, канонический объём видимости в Vulkan ограничен по глубине диапазоном $z \in [0, 1]$, а не $[-1, 1]$, как в OpenGL; отсюда иной вид третьей строки.

Все матрицы реализованы самостоятельно в файле `math.hpp`, без использования сторонних математических библиотек. Хранение — по столбцам, в том же порядке, в котором язык GLSL интерпретирует тип `mat4`.

2.3. Геометрия октаэдра

Правильный октаэдр — правильный многогранник с шестью вершинами, двенадцатью рёбрами и восемью гранями, каждая из которых является равносторонним треугольником. Наиболее удобное представление — вершины на осях координат: $(\pm 1, 0, 0)$, $(0, \pm 1, 0)$, $(0, 0, \pm 1)$. При таком выборе все рёбра равны $\sqrt{2}$, а центр фигуры совпадает с началом координат, что удобно для поворотов.

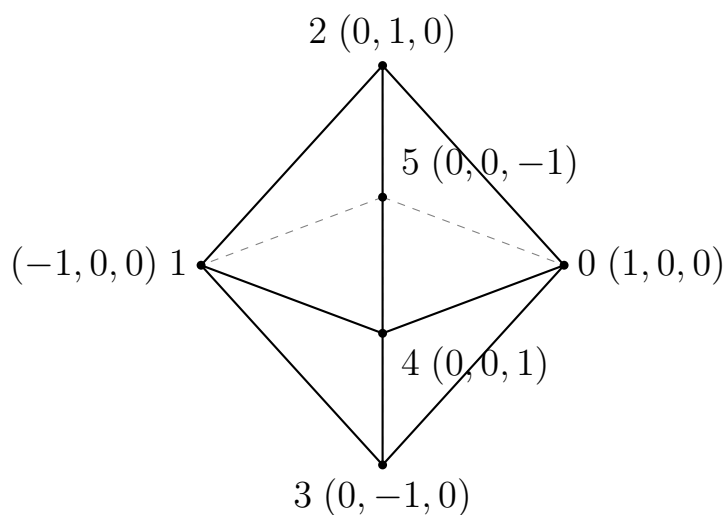


Рисунок 1 – Вершины октаэдра и их номера в вершинном буфере. Пунктиром показаны рёбра, сходящиеся в дальней вершине

Грани задаются индексным буфером: восемь треугольников, по четыре при верхней и нижней вершинах (таблица 3). Использование индексного буфера позволяет хранить шесть вершин вместо двадцати четырёх: каждая вершина принадлежит четырём граням и описывается один раз.

Таблица 3 – Грани октаэдра и направления их внешних нормалей

Индексы	Нормаль	Индексы	Нормаль
2, 4, 0	$(+1, +1, +1)$	3, 0, 4	$(+1, -1, +1)$
2, 1, 4	$(-1, +1, +1)$	3, 4, 1	$(-1, -1, +1)$
2, 5, 1	$(-1, +1, -1)$	3, 1, 5	$(-1, -1, -1)$
2, 0, 5	$(+1, +1, -1)$	3, 5, 0	$(+1, -1, -1)$

Для правильного изображения выпуклого тела используются два независимых механизма. *Буфер глубины*: вместе с цветом для каждого пикселя хранится глубина ближайшего записанного фрагмента, а фрагмент, оказавшийся дальше, отбрасывается. *Отсечение нелицевых граней*: грань, обращённая от наблюдателя, не растеризуется вообще; лицевая сторона определяется по знаку площади треугольника в координатах кадра, то есть по направлению обхода его вершин.

Порядок вершин в каждой тройке выбран так, чтобы обход был против

часовой стрелки при взгляде снаружи. Это проверяется по знаку скалярного произведения нормали грани на радиус-вектор её центра: для всех восьми граней произведение положительно, то есть нормали направлены наружу. Инверсия оси Y в матрице проекции переводит мировую систему координат, где ось Y направлена вверх, в систему координат кадра, где она направлена вниз, поэтому направление обхода в терминах Vulkan сохраняется, и в конвейере задано `frontFace = VK_FRONT_FACE_COUNTER_CLOCKWISE`.

2.4. Используемые объекты Vulkan

Для достижения цели работы создаются следующие объекты.

- `VkShaderModule` — два модуля, вершинный и фрагментный шейдеры, загружаемые из скомпилированных файлов SPIR-V. После создания конвейера уничтожаются, так как больше не нужны.
- `VkPipelineLayout` — описывает интерфейс между приложением и шейдерами; в данной работе содержит единственный диапазон push-констант.
- `VkPipeline` — графический конвейер. Существенные настройки: формат вершины из двух величин `VK_FORMAT_R32G32B32_SFLOAT` (позиция и цвет); примитив — список треугольников; отсечение нелицевых граней с обходом против часовой стрелки; включённые тест и запись глубины с функцией сравнения «меньше». Область вывода и прямоугольник отсечения объявлены динамическими и задаются при записи команд, поэтому при изменении размера окна конвейер не требуется пересоздавать.
- `VkBuffer` — вершинный и индексный буферы, размещённые через распределитель памяти `VmaAllocator` в памяти, доступной для записи со стороны процессора. Для объёма в несколько сотен байт промежуточный буфер в памяти видеокарты не требуется.
- `VkRenderPass` и `VkFramebuffer` — проход отрисовки с цветовым вложением и вложением глубины; предоставлены стартовым кодом.

Матрица (4) и цвет передаются в шейдер через push-константы — небольшой блок данных, записываемый непосредственно в командный буфер. Его размер составляет 80 байт (матрица 4×4 из чисел одинарной точности и четырёхкомпонентный цвет), что укладывается в гарантируемые спецификацией 128 байт. Такой способ не требует ни буферов равномерных данных, ни пулов дескрипторов.

2.5. Интерфейс, анимация и цвет

Окно интерфейса разделено на четыре раздела, соответствующих дополнительным заданиям: выбор проекции, преобразования фигуры, анимация и цвет. Значения ползунков разделов **Transform** и **Animation** используются для построения матрицы модели (5): таким образом, задания 2 и 3 сводятся к разным способам собрать M .

Траектория движения (задание 3) — вертикальная восьмёрка, намотанная на окружность:

$$x(t) = R \cos t, \quad y(t) = H \sin 2t, \quad z(t) = R \sin t, \quad (9)$$

где R — радиус окружности, H — амплитуда вертикальных колебаний. Удвоенная частота по оси Y даёт два подъёма за один оборот, то есть траектория не является плоской кривой. Параметр t наращивается на величину $\Delta t \cdot k$, где Δt — длительность кадра, а k — задаваемая в интерфейсе скорость; при паузе наращивание прекращается. Независимо от движения по траектории фигура вращается вокруг собственной оси с задаваемой угловой скоростью.

Цвет вершины (задание 5) вычисляется из её позиции в локальной системе координат при создании вершинного буфера: координата из диапазона $[-1, 1]$ переводится в канал цвета из диапазона $[0, 1]$ по формуле $c = 0,5p + 0,5$. Выбранный в интерфейсе цвет (задание 4) умножается на цвет вершины в вершинном шейдере, а интерполяция между вершинами выполняется растеризатором. Флажок `Procedural vertex colors` заменяет цвета вершин белым, и тогда фигура окрашивается только выбранным цветом.

```
1 #version 450
2
3 layout(location = 0) in vec3 in_position;
```

```

4 layout(location = 1) in vec3 in_color;
5
6 layout(location = 0) out vec3 frag_color;
7
8 layout(push_constant) uniform PushConstants {
9     mat4 mvp;    // модель * вид * проекция
10    vec4 tint;    // rgb — цвет из интерфейса, w — использовать ли цвета вершин
11 } pc;
12
13 // Выполняется для каждой вершины: переводит её в координаты экрана
14 // и вычисляет цвет, который дальше интерполируется вдоль граней.
15 void main() {
16     gl_Position = pc.mvp * vec4(in_position, 1.0);
17
18     // Доп. задания 4 и 5: цвет вершины задан процедурно от её позиции
19     // в локальной системе координат, а цвет из интерфейса умножается на него.
20     // Если tint.w равен нулю, цвета вершин заменяются белым
21     vec3 base = mix(vec3(1.0), in_color, pc.tint.w);
22     frag_color = base * pc.tint.rgb;
23 }

```

Листинг 1: Вершинный шейдер: преобразование вершины и расчёт цвета

3. Результаты

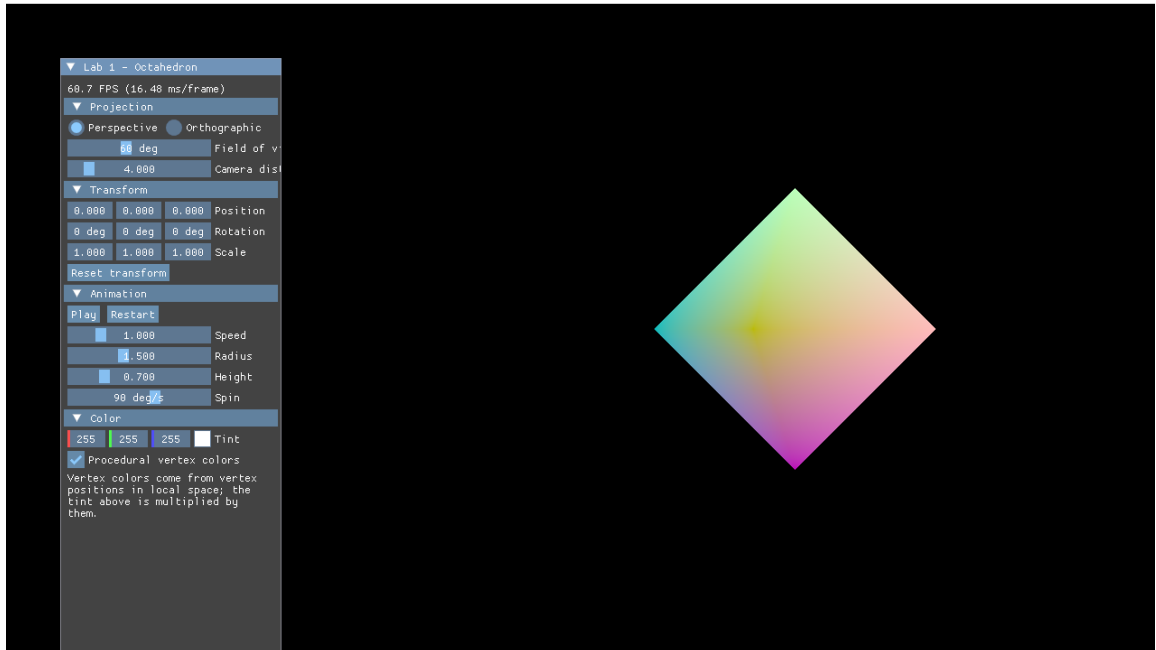


Рисунок 2 – Октаэдр в перспективной проекции. Цвета вершин заданы процедурно от их позиции в локальной системе координат и интерполируются вдоль граней

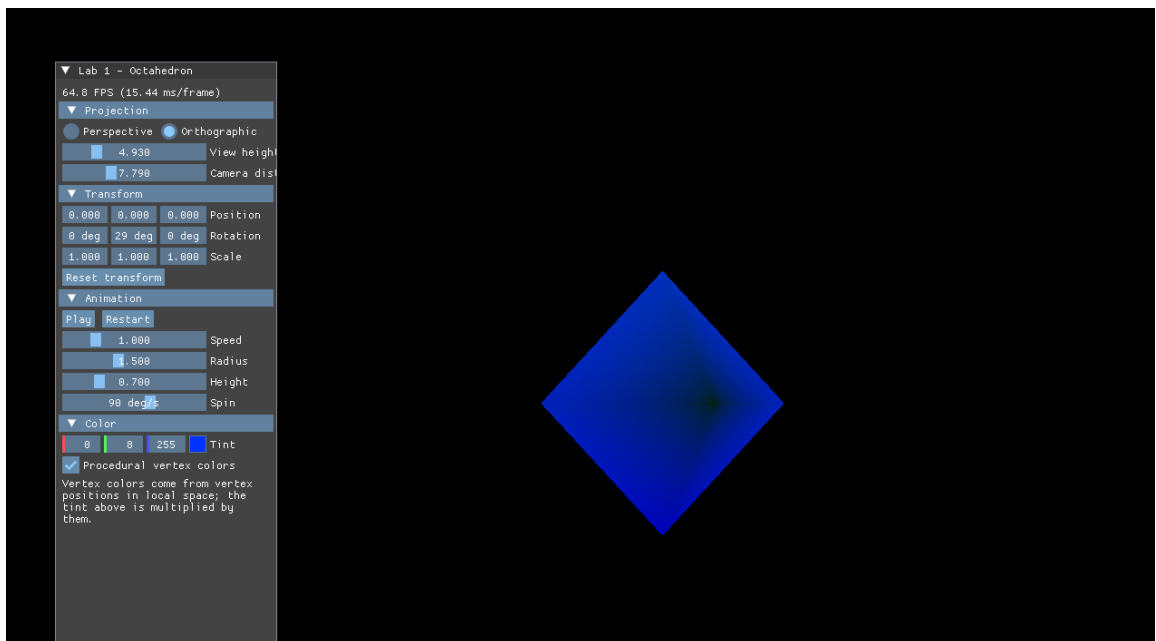


Рисунок 3 – Та же сцена в ортографической проекции: параллельные рёбра не сходятся, размер фигуры не зависит от удаления камеры

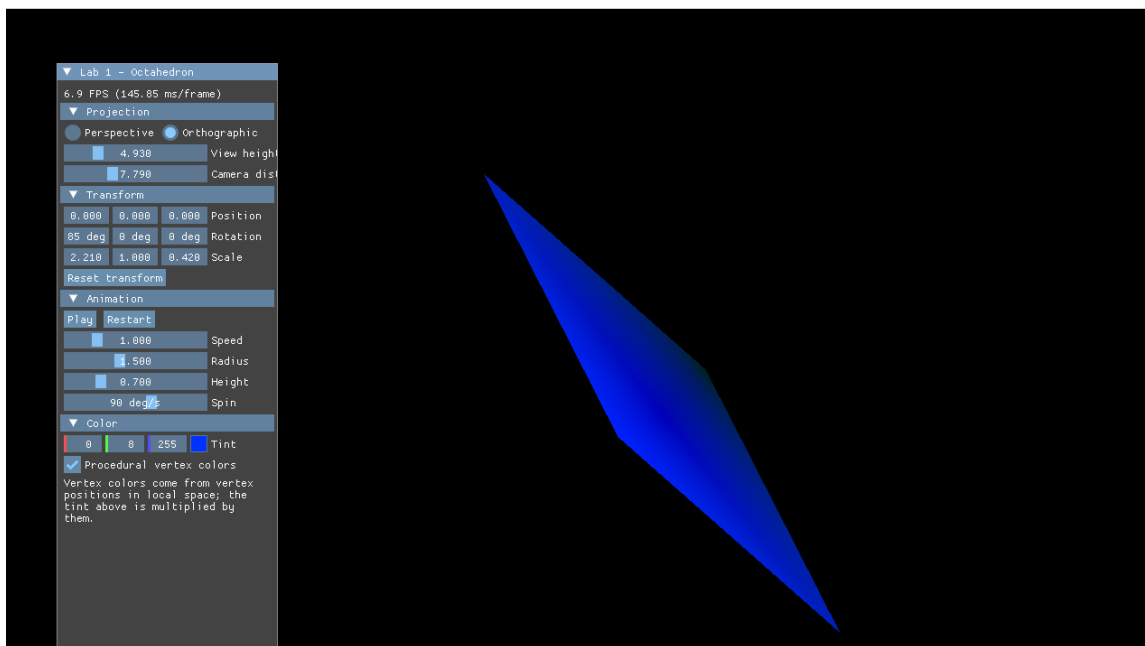


Рисунок 4 – Изменение позиции, поворота и растяжения фигуры ползунками раздела **Transform**

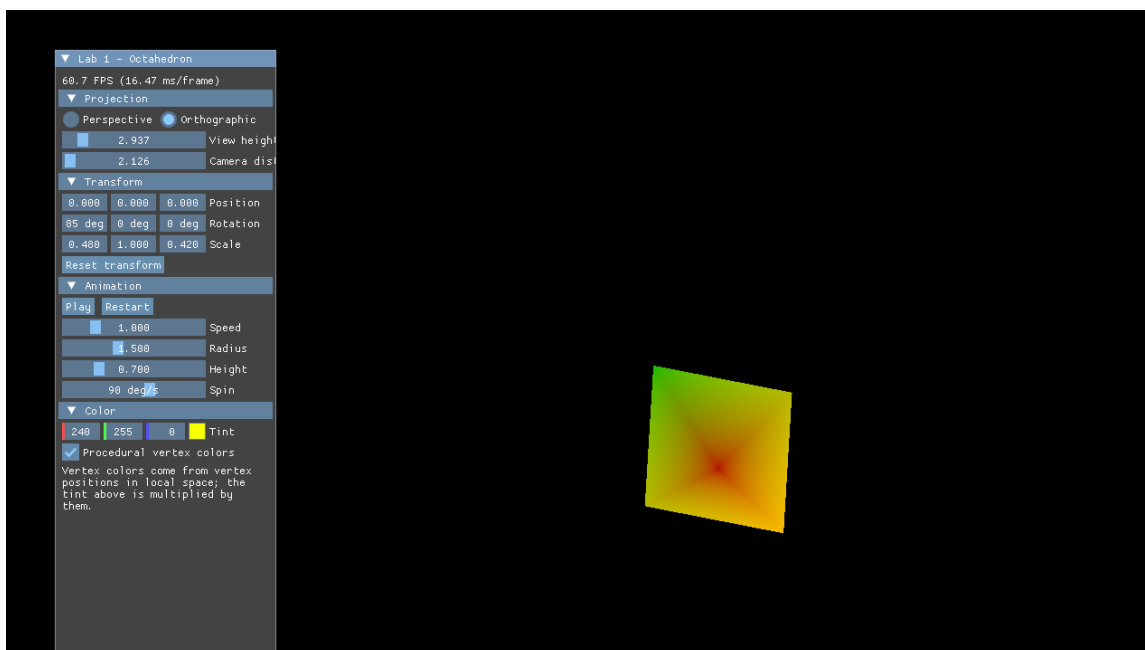


Рисунок 5 – Изменение цвета фигуры: выбранный в ColorEdit цвет умножается на цвета вершин

Различие проекций наиболее заметно, если сместить фигуру от центра экрана и увеличить её: в перспективной проекции грани, расположенные дальше от камеры, сокращаются, а параллельные рёбра сходятся; в ортографической проекции параллельность рёбер сохраняется, и размер фигуры не

зависит от удаления камеры.

4. Выводы

В ходе работы я научился строить простые трёхмерные объекты и выводить их на экран средствами Vulkan в реальном времени, работать с матрицами аффинных преобразований и проекций, а также пользоваться буфером глубины и отсечением нелицевых граней для правильного изображения выпуклого тела.

Главное, что я понял: представление точек в однородных координатах позволяет описать перенос, поворот, масштабирование, видовое преобразование и проекцию объектами одного типа — матрицами 4×4 . Благодаря этому вся последовательность преобразований сворачивается в одну матрицу, вычисляемую один раз за кадр, а каждая вершина обрабатывается единственным умножением. Практическим следствием стало то, что каждое из дополнительных заданий свелось к изменению одной из матриц, а код отрисовки, шейдеры и буферы остались неизменными: переключение проекции — это выбор функции, строящей матрицу P , а преобразования и анимация — разные способы собрать матрицу M .

Отдельно отмечу особенности соглашений Vulkan, которые пришлось учесть: диапазон глубины $[0, 1]$ вместо $[-1, 1]$ и направленная вниз ось Y в координатах кадра. Обе матрицы проекции содержат поправку на это; формулы, взятые из литературы по OpenGL без такой поправки, дали бы перевёрнутое изображение и неверную работу буфера глубины.

Полезным оказался и опыт отладки отсечения граней: ошибка в выборе лицевой стороны не бросается в глаза, поскольку выпуклая фигура остаётся похожей на себя — вместо ближних граней просто отображаются дальние. Обнаружить это удалось только по цветам вершин, так как в центре фигуры оказался цвет дальней вершины вместо ближней.